IMPLEMENTATION PLAN & ARCHITECTURAL DESIGN

Aspect-Based Hotel Review Analysis (Opinion Mining Enhancement)

Compiler Design Project – Team 17 | G1 Slot

Tanay Mohta (24BDS0294) | Aditya Rawat (24BCE2992)

1. Abstract

Customer reviews in the hospitality sector often contain opinions about multiple aspects of a hotel in the same sentence. For example, a guest may like the staff but be unhappy with the room. Traditional opinion mining usually gives the entire review one overall positive or negative score, which can hide these important details.

This project proposes **an** Aspect-Based Hotel Review Analysis System based on a classical compiler design approach. The system processes each review through multiple stages: lexical analysis, parsing, semantic analysis, and sentiment scoring using SentiWordNet**.**

The system focuses on six main hotel categories: Room, Staff, Service, Food, Cleanliness, and Facilities**.** It can split compound sentences, such as those containing *"but"*, into separate clauses so that each part can be analyzed independently. It also handles words that affect sentiment, such as negations (*"not"*, *"never"*) and intensifiers (*"very"*, *"extremely"*).

The final result is stored in a structured format containing the category, opinion, and sentiment, which can then be stored in a database and displayed through an administrative dashboard. The compiler-based approach makes the processing stages clear, modular, and easy to verify.

Objective. Extend the existing hotel opinion-mining system from overall positive/negative classification to aspect-based analysis. The enhanced system identifies the hotel aspect being discussed, extracts the associated issue or opinion, and determines the sentiment expressed toward that aspect.

Scope. The implementation focuses on six hotel-related aspects: Room, Staff, Service, Food, Cleanliness, and Facilities. A single review sentence may produce more than one aspect-level result when it contains multiple opinions.

2. Introduction

1. Background & Problem Context

Online customer reviews are an important source of feedback for hotels. Guests often comment on different parts of their experience, such as the room, staff, service, food, and cleanliness. However, these reviews are usually written as unstructured text, making it difficult to extract useful information from them.

Traditional opinion-mining systems usually classify an entire review or sentence as either positive or negative. Although this can show the overall sentiment, it can miss important details when a customer has mixed opinions.

For example:

"The room was not clean, but the staff were very helpful."

A system that gives the sentence only one overall sentiment may classify it as mixed or neutral. However, the review actually contains two separate opinions: the customer is unhappy with the room's cleanliness but happy with the staff.

Therefore, the proposed system uses Aspect-Based Sentiment Analysis (ABSA) to identify the specific hotel aspect being discussed and determine the sentiment associated with that aspect. This provides more detailed and useful feedback for hotel management.

2. The Compiler Design Perspective

The system is designed using concepts from classical compiler architecture. A compiler normally processes source code through different stages such as lexical analysis, syntax analysis, semantic analysis, and intermediate code generation.

In this project, a hotel review is treated as the input or "source program", and the same idea is applied to process the review step by step.

- Lexical Analysis: The review is broken into tokens and words are identified as aspects, opinion words, negations, intensifiers, conjunctions, and other relevant terms.
- Syntax Analysis: The tokens are checked using a grammar to understand the structure of the review. Compound sentences can be divided into separate clauses, especially when words such as *"but"* are used.
- Semantic Analysis: The system determines which opinion belongs to which hotel aspect. It also handles words such as *"not"* and *"very"*, which can change the meaning or strength of the sentiment.
- Sentiment Evaluation: SentiWordNet is used to obtain sentiment scores for opinion words. These scores are adjusted based on relevant modifiers such as negation and intensifiers.
- Intermediate Representation (IR): The result is converted into a structured record:

$$\text{IR Tuple} = \langle \text{Review ID, Category, Issue/Opinion, Opinion Term, Polarity, Score} \rangle$$

- Output Generation & Storage: The structured results are stored in a database and can be used to generate category-wise reports for the hotel administrator.

This approach keeps the analysis structured, modular, and easy to understand, while allowing each stage of the processing pipeline to be examined separately.

2. Architectural Design

The proposed architecture treats a review sentence as the source input to a compiler-style analysis pipeline. The system is organized into distinct stages so that lexical processing, syntactic structure, semantic interpretation, sentiment evaluation, and final result generation remain clearly separated.

High-level flow

Input Review → Lexical Analysis → Syntax Analysis → Semantic Analysis → Sentiment Evaluation → Intermediate Representation → Structured Output

2.1 System Components

Component	Responsibility	Primary output
Input Handler	Accepts the hotel review sentence and performs basic input validation/normalization.	Raw review text
Lexical Analyzer	Tokenizes the review and recognizes categories such as aspect terms, opinion words, negations, intensifiers, conjunctions, and punctuation.	Token stream
Syntax Analyzer	Parses the token stream using the project grammar and identifies clause-level structure, including multiple clauses connected by words such as “but”.	Parse structure / parse tree
Semantic Analyzer	Associates opinion expressions with hotel aspects and determines the semantic effect of negation and intensifiers.	Aspect-opinion records
Sentiment Engine	Uses SentiWordNet polarity information to assign positive, negative, or neutral sentiment and a corresponding score.	Polarity + score
Intermediate Representation	Stores each clause/result in a consistent structured form for downstream output and storage.	Aspect-level tuples
Output & Reporting	Displays the aspect-wise result and overall sentiment in a readable report for the hotel administrator.	Structured report

2.2 Processing Flow

1. Input: A user provides a hotel review sentence.

2. Lexical Analysis: The sentence is converted into meaningful tokens, including aspect words, opinion words, negations, intensifiers, and conjunctions.

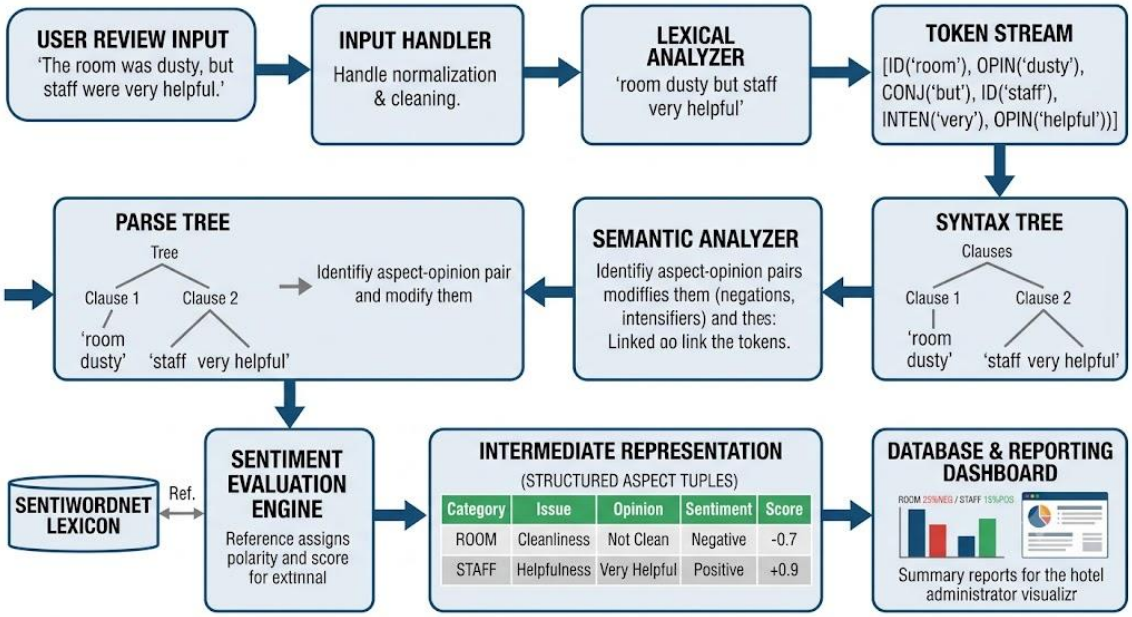
3. Syntax Analysis: Tokens are grouped into clauses and phrases according to the defined grammar. This allows sentences containing multiple aspects to be represented separately.

4. Semantic Analysis: The system determines which aspect an opinion belongs to and applies the effect of modifiers such as “not”, “never”, “very”, or “extremely”.

5. Sentiment Evaluation: The identified opinion words are matched against the SentiWordNet resource and their polarity is adjusted using the semantic information.

6. Intermediate Representation: Each aspect-level result is represented as a structured record such as (Room, Cleanliness, Not Clean, Negative, score).

7. Output: The final aspect-wise results and overall sentiment are presented to the administrator.



3. Design Rationale

3.1 Why a compiler-style architecture? The project is intended for a Compiler Design course. Separating lexical, syntax, semantic, and output stages provides a direct mapping from classical compiler phases to the review-analysis problem while keeping each stage independently testable.

3.2 Why aspect-based analysis? The existing system provides only a single overall polarity. This loses information when a review contains multiple hotel concerns. Aspect-based analysis retains the positive or negative sentiment while also identifying whether the feedback concerns the room, staff, service, food, cleanliness, or facilities.

3.3 Why retain SentiWordNet? SentiWordNet is part of the existing project concept, so retaining it preserves continuity with the base system. It provides a lexical basis for sentiment scoring, while the proposed semantic stage supplies additional context for negation and modifiers.

3.4 Why allow multiple aspect results? A realistic hotel review may contain several independent opinions in one sentence. Treating each relevant clause as a separate aspect-level result produces more useful information than forcing the whole sentence into one positive or negative label.

3.5 Why keep the architecture modular? A modular design allows the team to implement and validate one phase at a time. It also limits debugging scope: a tokenization issue can be isolated from parsing, semantic rules, or sentiment scoring.

4. Implementation Plan

Implementation will follow an incremental sequence. Each phase produces a working component that can be tested before the next phase is added.

Phase	Implementation focus	Deliverable	Exit criterion
Phase 1 – Requirements & Vocabulary	Freeze the six aspect categories; define common issue/opinion labels; define polarity values and output fields.	Aspect taxonomy, opinion lexicon, output specification	All six categories and expected outputs documented
Phase 2 – Lexical Analysis	Implement token recognition for words, aspect terms, opinion terms, negations,	Working lexer and token stream	Representative test sentences tokenize

Phase	Implementation focus	Deliverable	Exit criterion
	intensifiers, conjunctions, and punctuation.		correctly
Phase 3 – Syntax Analysis	Implement the review grammar and parse common structures, including single-clause and two-clause reviews joined by contrast/conjunction terms.	Parser and parse representation	Target test set parses successfully without ambiguity
Phase 4 – Semantic Analysis	Attach opinions to aspects, detect negation and intensifier effects, and separate multiple aspect-bearing clauses.	Aspect-opinion semantic records	Examples such as “not clean”, “very helpful”, and “dirty but helpful” produce correct semantic attributes

Phase	Implementation focus	Deliverable	Exit criterion
Phase 5 – Sentiment Evaluation	Integrate SentiWordNet lookup, compute aspect-level polarity/score, and define the final score convention.	Aspect sentiment evaluator	Positive, negative, neutral, and negated examples produce consistent polarity
Phase 6 – IR & Output	Convert results into a consistent intermediate representation and generate a human-readable report.	Structured aspect-wise output	Single-sentence demo produces category, issue/opinion, sentiment, and score
Phase 7 – Database & Reporting	Persist review text and aspect-level results and provide administrator-oriented summaries.	Database records and summary view	Stored results can be retrieved and aggregated by category
Phase 8 – Integration & Testing	Integrate all phases, test end-to-end behavior, document limitations, and prepare demonstration cases.	Final prototype and test report	Complete pipeline runs from input to final output

5. Data and Output Design

The core result should be represented consistently regardless of the sentence being analyzed. The following fields are sufficient for the project scope:

Field	Purpose
Review ID	Unique identifier for the analyzed review
Review Text	Original review sentence
Category	One of Room, Staff, Service, Food, Cleanliness, Facilities
Issue / Opinion	Specific issue or opinion extracted from the sentence
Opinion Term	The main sentiment-bearing word/phrase
Polarity	Positive, Negative, or Neutral
Score	Numeric sentiment score derived from the sentiment evaluation stage

6. Example End-to-End Result

Input: “The room was not clean, but the staff were very helpful.”

Category	Issue / Opinion	Opinion	Sentiment	Score
Room	Cleanliness / Not Clean	Not clean	Negative	-
Staff	Helpfulness	Very helpful	Positive	+

Overall result: Mixed sentiment

7. Testing and Validation Plan

Test area	Test cases	Expected evidence
Lexical tests	Aspect words, opinion words, negations, intensifiers, conjunctions, punctuation	Token output matches expected token classes
Syntax tests	Single-clause reviews, multiple clauses, contrast using "but"	Expected parse structure is produced
Semantic tests	Negated and intensified opinions; multiple aspects	Correct aspect-opinion association and modifier handling
Sentiment tests	Clearly positive, negative, neutral, and mixed examples	Expected polarity class and score direction

7. Testing and Validation Plan (continued)

Test area	Test cases	Expected evidence
End-to-end tests	Representative hotel reviews	Correct final aspect-wise report from raw sentence
Persistence tests	Store and retrieve analyzed reviews	No loss of review or aspect-level result data

8. Scope Boundaries and Risk Control

Controlled vocabulary scope. The initial implementation will focus on six predefined hotel aspects and a manageable set of opinion labels. This keeps the project demonstrable and reduces unnecessary rule complexity.

Ambiguous language. Some expressions may not map cleanly to a single aspect. These cases will be handled conservatively and recorded as limitations rather than expanding the grammar indefinitely.

Negation and modifiers. Negation such as "not clean" and modifiers such as "very helpful" will be treated explicitly because they can change the interpretation of the underlying sentiment word.

Multiple opinions. The system will allow multiple aspect-level results from one review instead of collapsing every sentence into a single label.

Implementation discipline. Each phase will be completed and tested before integration with the next phase. This is intended to keep the project finishable within the course timeline.

9. Expected Final Deliverable

The completed prototype will accept a hotel review sentence and produce a structured aspect-wise analysis showing: (1) the hotel category discussed, (2) the extracted issue or opinion, (3) its polarity, and (4) a sentiment score. The same result structure will be stored so that the administrator can review and summarize feedback category-wise.